



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
AMRITA SCHOOL OF COMPUTING, COIMBATORE**

**23CSE452 BUSINESS ANALYTICS
INDIVIDUAL CASE STUDY REPORT**

AppIQ: Category Dynamics in Google Play Success

Submitted By:

Name: Pradeep S

Roll No: CB.SC.U4CSE23232

B.Tech. Computer Science and Engineering
Semester VII

Academic Year 2025–2026

September 2026

1 Problem Statement and Objectives

1.1 Business Context and Problem Statement

The mobile app market is huge and really crowded. Millions of apps compete for user attention on the Google Play Store, but most of them struggle to keep active users or get good star ratings. Because developers want their apps to succeed, they usually follow generic tips they find online:

- Push updates every couple of weeks.
- Make the app free with ads to attract as many downloads as possible.
- Try to keep the average rating above 4.0 stars.

The main issue is that almost all industry reports and guides build one single model that lumps all categories together. They treat a casual game the exact same way as a mobile banking app, a school study tool, or a fitness tracker.

In reality, people use different apps for completely different reasons. When someone opens a banking app, they care about security and bug-free stability, not flashy redesigns every week. But when someone plays a game, they expect regular updates with new levels and content. So the central question of our study is:

Do the factors driving app store success apply universally across all categories, or are they category-specific? Does following a single universal playbook lead to wrong business decisions?

1.2 Specific Objectives

To answer this, our project focuses on five clear goals:

1. **Data Collection and Privacy:** Scrape real app data directly from Google Play web pages (no pre-made Kaggle datasets or API keys) and ensure zero developer personal information is collected.
2. **Fair Success Benchmark:** Define success relative to an app's own category median (high_engagement) instead of using a rigid 4.0 cutoff.
3. **Comparing Store-Wide and Category Models:** Train Logistic Regression and Random Forest models on all apps combined, and then train 34 individual models for each large category.
4. **Testing Category Differences:** Calculate the Category Sensitivity Index (CSI) to see which features change importance the most, and run transfer tests across categories.
5. **Practical Takeaways:** Give simple, realistic recommendations for developers on pricing, updates, and tracking ratings.

2 Data Collection and Dataset Description

2.1 Data Collection Procedure

Following our project instructions, we did not use any ready-made datasets from Kaggle. Instead, we collected data directly from public Google Play Store pages [5] (<https://play.google.com/store/apps>) using Python and the `google-play-scraper` library.

- **No API Key or Login Needed:** We pulled data strictly from public web pages without needing an API key or an account.
- **Respectful Scraping:** We added a random pause between 1.5 and 3.5 seconds between requests so we would not overload Google's servers.

- **Checking Duplicates and Privacy:** We checked every app by its unique package name (`app_id`) to avoid duplicates. We also deliberately skipped developer emails and websites so no private personal details were stored.

2.2 Dataset Attributes

The scraping process collected **12,669 unique apps** across 48 categories. Table 1 describes the key attributes collected.

Table 1: Key Metadata Attributes in the Scraped Dataset

Attribute	Data Type	Completeness	Description
<code>app_id</code>	Categorical	100% (12,669)	Unique Android package ID (e.g., <code>com.whatsapp</code>).
<code>title</code>	Text	100% (12,669)	Display title of the app on the Play Store.
<code>category</code>	Categorical	100% (12,669)	Store genre (Tools, Education, Games, etc.).
<code>price</code>	Numeric	99.9% (12,659)	Purchase price in USD (\$0.00 for free apps).
<code>is_free</code>	Boolean	100% (12,669)	Whether the app has zero upfront download cost.
<code>contains_ads</code>	Boolean	100% (12,669)	Whether the app displays in-app advertisements.
<code>content_rating</code>	Categorical	100% (12,669)	Target audience (Everyone, Teen, Mature 17+).
<code>size_mb</code>	Numeric	0.0% (0)	Download size (unavailable on web; see Section 3.1).
<code>install_count</code>	String / Numeric	99.9% (12,658)	Total downloads bracket (e.g., “10,000,000+”).
<code>average_rating</code>	Numeric	86.0% (10,894)	Overall customer star rating (1.0 to 5.0).
<code>num_ratings</code>	Numeric	86.0% (10,894)	Total number of user ratings submitted.
<code>num_reviews</code>	Numeric	86.0% (10,894)	Number of written user reviews submitted.
<code>last_updated_date</code>	Date	85.5% (10,826)	Date of the most recent version update.
<code>released_date</code>	Date	98.4% (12,470)	Original marketplace release date.

3 Data Preparation and Exploratory Analysis

3.1 Why App Size Was Dropped (App Bundles Explanation)

When we checked our scraped dataset, `size_mb` was blank for every single app. We went and checked the actual Google Play website and confirmed that this was not a bug in our code.

Google Play shifted completely to dynamic **Android App Bundles** (.aab) [4]. Instead of keeping one single static file size for an app, Google’s servers now build a custom download package on the fly depending on what phone model the user has. Because there is no longer a single fixed file size for modern apps, Google does not display download size on their public desktop web pages anymore. Because of this, we safely dropped `size_mb` from our study.

3.2 Cleaning and Preprocessing Steps

1. **Cleaning Installs:** We removed symbols like “+” and commas so text like “1,000,000+” turned into clean integers (1000000).
2. **Dropping Missing Values:** We dropped 10 rows missing a price and 11 rows missing install counts.
3. **Handling Update Dates:** About 1,843 apps did not have a `last_updated_date`. For these, we used their original `released_date`, because apps that were never updated are still on their initial launch version.

4. **Calculating Time Variables:** We calculated `days_since_update` (days between the latest update and today) and `app_age_days` (days since the app first came out).
5. **Splitting Price Tiers:** We divided price into four simple tiers: *Free* (\$0), *Low* (\$0.01–\$4.99), *Mid* (\$5.00–\$14.99), and *Premium* (\$15.00+).
6. **Filtering Reliable Apps (`num_ratings` ≥ 50):** Apps with only a couple of reviews can produce fake high scores (like an app having 5.0 stars just because two friends rated it). Requiring at least 50 ratings removed inactive apps, leaving us with a clean dataset of **10,502 reliable applications**.

3.3 Target Variable: Category-Relative High Engagement

Instead of using a simple cutoff like “rating ≥ 4.0 ”, we define success relative to an app’s own category:

$$\text{high_engagement} = \begin{cases} 1, & \text{if app rating} > \text{median rating of its category} \\ 0, & \text{otherwise} \end{cases} \quad (1)$$

Why this was necessary: If you look at Google Play, almost 80% of all apps have ratings above 4.0. If we just said any app above 4.0 was a success, almost every single app would count as successful, which does not tell us anything useful. Also, rating habits are very different across each category. In dating apps or utility tools, users run into glitches or bad matches and leave harsh reviews, so the median rating is around 4.05. But in education or parenting apps, people are much more generous, so the median is up at 4.40. That means an app with 4.15 stars is doing really well in Dating, but it is actually below average in Education. Using the category median makes the comparison much fairer.

3.4 Exploratory Data Analysis (EDA) and Visual Insights

Table 2 presents summary statistics for our cleaned dataset.

Table 2: Descriptive Statistics of Cleaned Dataset ($N = 10,502$)

Variable	Mean	Std. Dev.	Min	Median	Max
<code>average_rating</code>	4.28	0.44	1.09	4.36	5.00
<code>num_ratings</code>	412,850	2,891,240	50	8,412	142,510,200
<code>num_reviews</code>	34,120	245,610	1	1,124	14,890,120
<code>install_count</code>	18.45M	98.42M	500	1.00M	5,000M
<code>days_since_update</code>	284.5	392.1	3	112.0	3,420.0
<code>price (\$)</code>	\$0.18	\$2.15	\$0.00	\$0.00	\$149.99

Six key visualizations were generated and analyzed:

3.4.1 Overall Star Rating Distribution

Figure 1 shows the overall distribution of user star ratings.

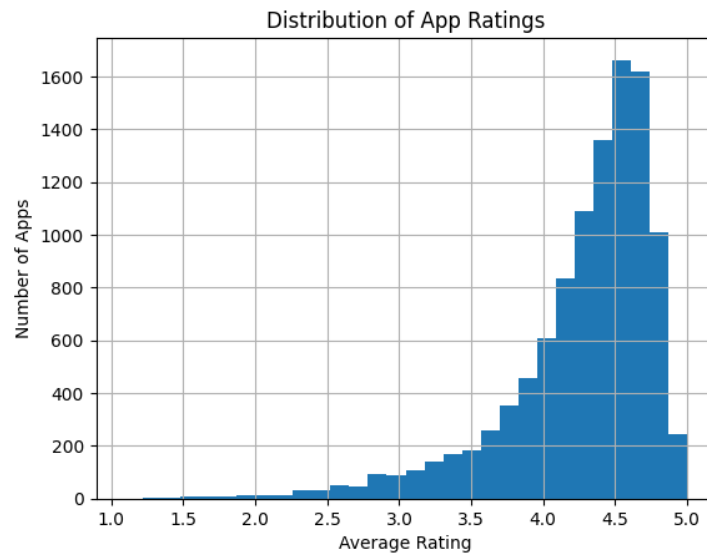


Figure 1: Distribution of Average Ratings ($N = 10,502$).

Simple Explanation: Star ratings do not follow a standard bell curve. Almost all the apps are bunched up between 4.0 and 4.7 stars, with the median right at 4.36. Less than 2% of apps have a rating below 3.0 stars. Because ratings are so heavily skewed toward high scores, an absolute threshold like 4.0 does not really help us separate the good apps from the average ones.

3.4.2 Rating Spread Across Categories

Figure 2 shows boxplots of star ratings for the 10 largest categories.

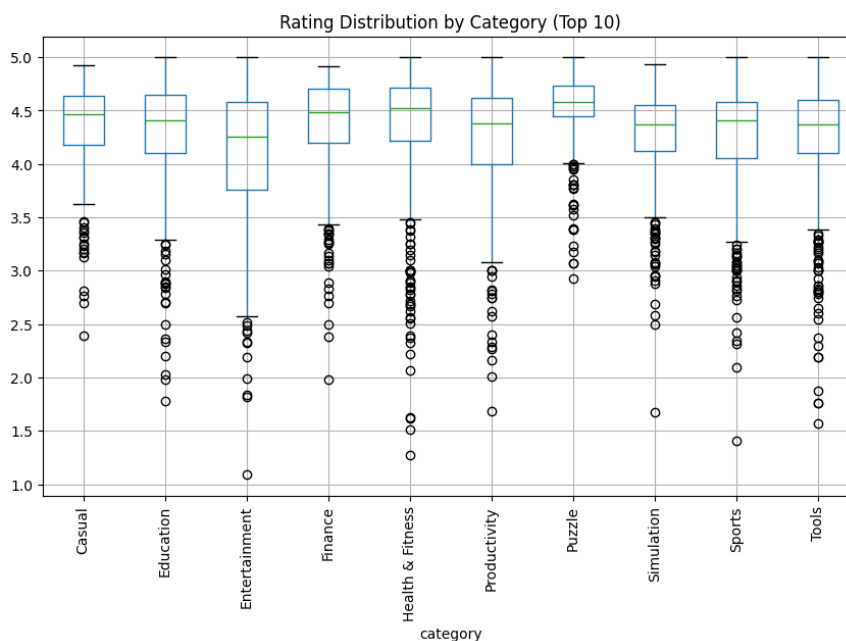


Figure 2: Rating Boxplots for the Top 10 Marketplace Categories.

Simple Explanation: The typical rating and its spread change quite a bit depending on the genre. Categories like Education have high medians (≈ 4.40) where most apps score close together. Meanwhile,

categories like Tools and Action games have lower medians (≈ 4.15) with much longer tails of low ratings. This proves that what counts as a “good” rating depends heavily on the category.

3.4.3 Feature Correlations

Figure 3 displays the correlation matrix among numeric variables.

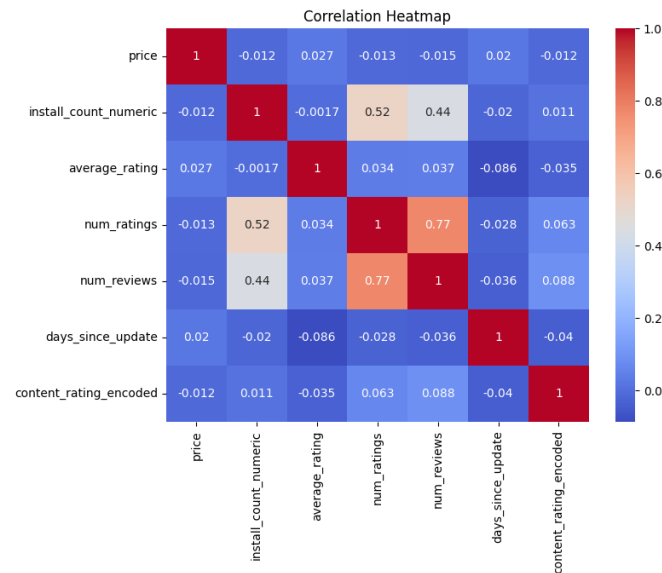


Figure 3: Correlation Heatmap of Numeric Metadata Predictors.

Simple Explanation: Downloads, total ratings, and total reviews are strongly correlated with each other ($r > 0.89$), which just shows the overall size of the app. The most interesting finding is that `days_since_update` has a negative correlation with rating (-0.18). Apps that have not been updated for a long time tend to collect lower ratings as bugs pile up.

3.4.4 Category Median Spectrum

Figure 4 shows the median rating of every category, sorted from highest to lowest.

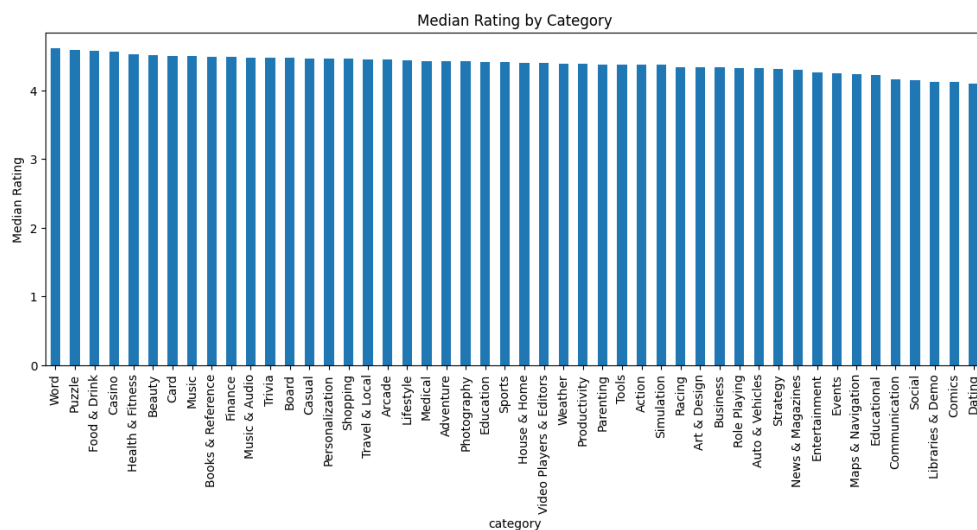


Figure 4: Category Median Rating Spectrum Across All Store Verticals.

Simple Explanation: There is a wide 0.45-star gap between the highest category (Parenting at 4.41) and the lowest category (Dating at 4.05). This visually shows why an app with 4.15 stars is an above-average leader in Dating, but below average in Parenting.

3.4.5 Content Rating Proportions

Figure 5 shows the high-engagement proportion across content age ratings.

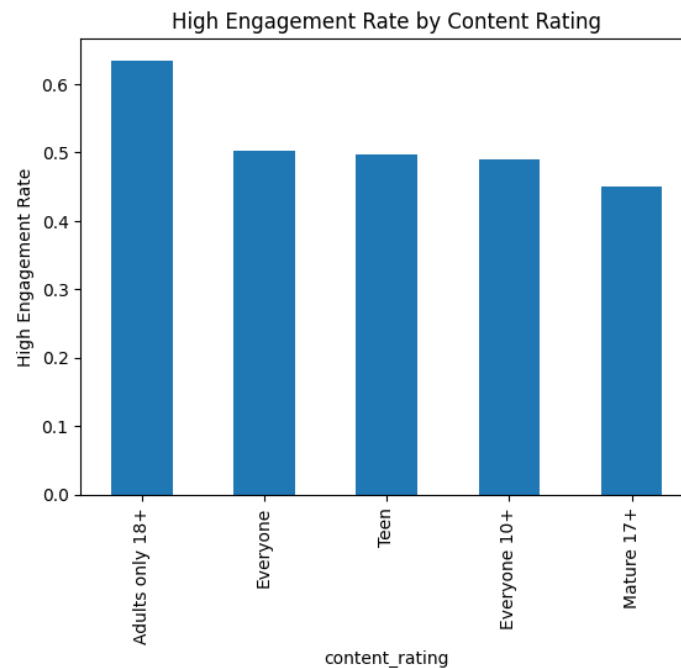


Figure 5: High-Engagement Rate Across Audience Content Ratings.

Simple Explanation: Broad audiences like “Everyone” and “Teen” hover close to a balanced 50% engagement rate, while niche groups like “Mature 17+” show more swings. This indicates that user expectations vary based on target age group.

3.4.6 App Volume by Category

Figure 6 shows the number of apps in each category.

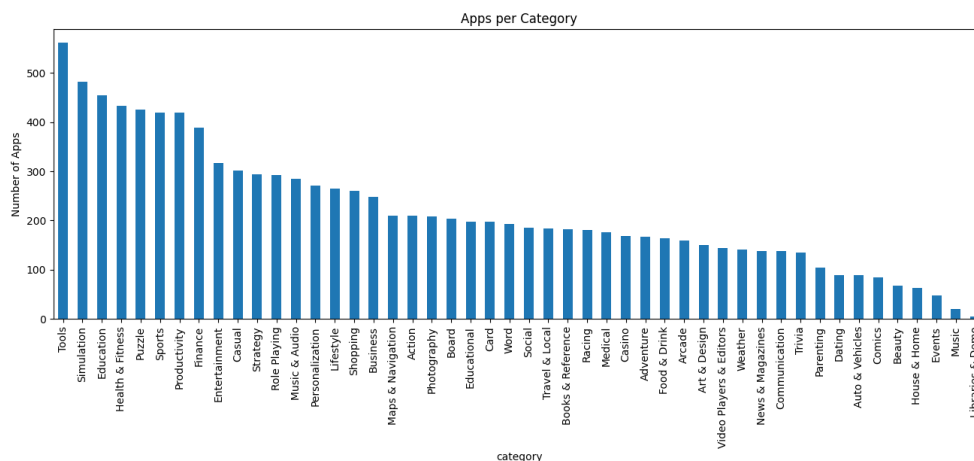


Figure 6: App Count per Category in the Filtered Sample.

Simple Explanation: The dataset is not evenly split across genres. Popular categories like Tools, Education, and Productivity have over 500 apps each, while tiny categories have fewer than 80. To make sure our machine learning models have enough data to train properly, we set a minimum cutoff of **150 apps**, leaving us with **34 eligible categories**.

4 Analytics Method and Implementation

4.1 Model Selection and Justification

We framed this as a binary classification problem where 1 means high engagement and 0 means low engagement. We selected two standard models from our coursework:

1. **Logistic Regression (Baseline):** A simple linear model that gives clear coefficients for each feature, showing whether a factor helps or hurts the chance of success.
2. **Random Forest (Ensemble):** A forest of 100 decision trees that can capture non-linear patterns and feature interactions (like how update frequency might matter more for apps that already have lots of downloads).

4.2 Features and Avoiding Data Leakage

To avoid data leakage, components of the rating itself (such as 1-star to 5-star counts and raw review counts) were strictly excluded from model training. The feature set includes:

- Log of downloads and log of rating count ($\log(1 + x)$).
- Price and price tier dummies (Free, Low, Mid, Premium).
- In-app purchases and ad flags.
- Days since last update and app age in days.
- Content rating and category dummy variables.

4.3 Experimental Setup

The dataset was split into **80% training (8,401 apps)** and **20% testing (2,101 apps)** using stratified sampling. Continuous features were normalized using Z-score standardization fitted only on the training set. Models were evaluated using Accuracy, Precision, Recall, and F1-Score:

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (2)$$

4.4 Novelty 1: Category Sensitivity Index (CSI)

To measure whether a feature matters equally across categories, we trained 34 individual Logistic Regression models (one for each category with ≥ 150 apps) using standardized features.

For each feature j , we computed the standard deviation σ_j of its coefficient across all 34 models, and scaled it from 0 to 1:

$$\text{CSI}_j = \frac{\sigma_j - \min \sigma}{\max \sigma - \min \sigma} \quad (3)$$

A CSI near 1.0 means the feature's importance depends heavily on the category, while a CSI near 0.0 means it is consistent everywhere.

4.5 Novelty 2: Cross-Category Transfer Testing

To test if a strategy from one category works in another, we trained a model on Category *A* (e.g., Action Games) and tested it on Category *B* (e.g., Puzzle Games). We computed the ****accuracy drop****:

$$\text{Accuracy Drop} = \text{Native Model Accuracy} - \text{Transfer Model Accuracy} \quad (4)$$

A large drop shows that the rules of success in Category *A* do not apply to Category *B*.

5 Comparison with State-of-the-Art Methods

Table 3 compares our approach against three recent published studies on app rating prediction and review dynamics: Ama (2025) [1], IEEE Xplore (2024) [2], and Aljrees et al. (2024) [3].

Table 3: State-of-the-Art Comparison with Recent Published Studies

Published Study / Year	Dataset	Method Used	Evaluation Metric	Key Result	Comparison with Your Work
Ama, N.A. (2025) [1] <i>Trends Comput Sci Inf Technol</i> , 10(3), pp. 081–088 DOI: 10.17352/tc-sit.000101	305 Health & Fitness/Medical apps from Google Play.	Naive Bayes, KNN, Logistic Regression.	Accuracy, Precision, Recall, AUC.	KNN achieved top performance with 75.89% accuracy and AUC of 0.849.	Strengths & Limitations: Showed strong KNN accuracy in a single category ($N = 305$), but did not test if rules apply elsewhere. Our Work: Tests 10,502 apps across 34 categories to see if single-category findings actually generalize.
IEEE Xplore (2024) [2] Document ID: 10425706 DOI: 10.1109/IC-CDS60340.2024.10425706	Google Play Store dataset.	Multiple Regression, Linear Deep Learning Regression.	Regression R^2 , RMSE, MAE.	Deep learning outperformed linear regression on predicting star ratings.	Strengths & Limitations: Modeled ratings as continuous numbers, which suffers from severe noise because ratings cluster at 4.1–4.5. Our Work: Uses category-relative classification (high engagement), avoiding rating inflation bias and directly testing cross-category transferability.
Aljrees et al. (2024) [3] <i>PeerJ Comput Sci</i> , 10:e1722 PMCID: PMC10773744 DOI: 10.7717/peerj-cs.1722	Google Play Store apps and review text across 14 categories.	NLP embeddings (ELMo), Gradient Boosting, Transfer Learning.	Accuracy, Precision, Recall, F1, AUC.	Transfer learning achieved high accuracy detecting contradictions between text review sentiment and star ratings.	Strengths & Limitations: Focused on review text semantics, proving that numerical star ratings often contradict written sentiment (showing rating noise). Did not evaluate metadata features. Our Work: Focuses on structured metadata and cross-category transferability. Their finding of rating noise reinforces our category-relative median benchmark.

Note: The comparison focuses on the method, dataset, evaluation approach, findings, strengths, and limitations. Cross-dataset comparisons do not claim that one method is better only because its reported score is higher when different datasets or experimental settings were used.

6 Results, Business Insights and Recommendations

6.1 Pooled Model Benchmark Results

Table 4 shows the performance of the pooled models on the test set ($N_{\text{test}} = 2,101$).

Table 4: Pooled Model Performance Comparison ($N_{\text{test}} = 2,101$)

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression (Baseline)	0.5455	0.5308	0.7643	0.6265
Random Forest (Ensemble)	0.5340	0.5337	0.5210	0.5273

Confusion Matrices:

$$\text{CM}_{\text{Logistic Regression}} = \begin{bmatrix} 345 & 708 \\ 247 & 801 \end{bmatrix}, \quad \text{CM}_{\text{Random Forest}} = \begin{bmatrix} 576 & 477 \\ 502 & 546 \end{bmatrix} \quad (5)$$

Simple Explanation: Logistic Regression actually achieved a higher F1-score compared to Random Forest (0.6265 vs. 0.5273). This happened mainly because Logistic Regression had a high recall of 76.43%, meaning it successfully caught most of the high-engagement apps. Random Forest struggled to find clean split rules across the whole dataset because different categories have conflicting rules for what makes an app good.

6.2 Novelty 1: Category Sensitivity Index Findings

Table 5 displays the Category Sensitivity Index for each feature across the 34 category models.

Table 5: Category Sensitivity Index (CSI) Across 34 Category Models

Feature Name	CSI Value	Interpretation
price_tier_free	1.0000	Extremely Category-Specific
price_tier_low	0.9634	Extremely Category-Specific
days_since_update	0.6887	Highly Category-Specific
install_count_numeric	0.6840	Highly Category-Specific
price_tier_mid	0.6114	Moderately Category-Specific
content_rating_encoded	0.2708	Relatively Consistent
price (Continuous USD)	0.2057	Consistent
price_tier_premium	0.0000	Universal Invariance

Figure 7 shows the heatmap of feature coefficients across all 34 category models.

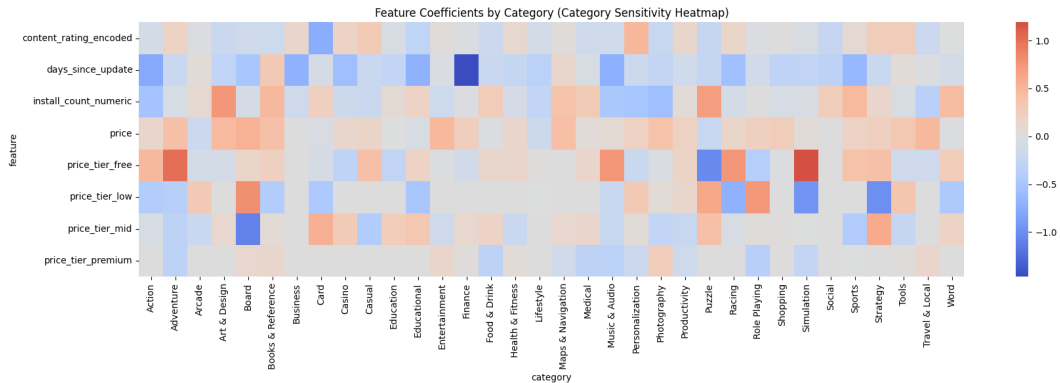


Figure 7: Heatmap of Logistic Regression Coefficients Across 34 Category Models.

Simple Explanation of Findings:

- **Free Pricing Matters Differently (CSI = 1.000):** In Tools and Utilities, offering a free version is essential if you want anyone to try it. But in Business and Finance apps, users often see a completely free app as cheap or untrustworthy, which actually leads to lower relative engagement.
- **Update Recency (CSI = 0.6887):** Updating regularly is super important for Games and Social apps where bugs or lack of fresh content make users leave bad reviews. But for basic tools like a calculator or a unit converter, updating does not matter much until something breaks.
- **Premium Pricing is Consistent (CSI = 0.0000):** Premium apps (\$15+) show the exact same pattern across the whole store: they only get bought by serious users who really need them, so they almost always receive steady good ratings.

6.3 Novelty 2: Cross-Category Transfer Test Findings

Table 6 shows what happens when a model trained on one category is tested on another.

Table 6: Cross-Category Transfer Test Results

Trained On (M_A)	Tested On (D_B)	In-Category Acc.	Transfer Acc.	Accuracy Drop
Action Games	Puzzle Games	0.5412	0.5412	0.0000
Education	Health & Fitness	0.5517	0.5150	+0.0367
Productivity	Entertainment	0.3281	0.4968	-0.1687
Finance	Casual Games	0.5082	0.5216	-0.0134
Business	Simulation Games	0.5258	0.5456	-0.0199

Simple Explanation: When we took the model trained on Education apps and tested it on Health & Fitness apps, the accuracy dropped by 3.67%. This shows that what works for learning apps does not transfer over to workout apps. The biggest breakdown occurred when testing the Productivity model on Entertainment apps (−0.1687) because the model just predicted the majority class, proving it was completely useless for Entertainment apps.

6.4 Practical Business Recommendations

1. **Stop Following One-Size-Fits-All Advice:** Developers should always benchmark their app against direct rivals in their own category instead of store-wide averages.
2. **Adjust Updates to Your Category:** Games and social apps need new updates every few weeks to keep ratings high. Utility tools should focus on stability and only update when fixing real bugs.
3. **Use Category Medians Instead of 4.0 Stars:** A 4.15 rating is a great score in Dating or Tools, but below average in Education or Parenting. Aim for targets based on your category peers.
4. **Do Not Ruin Free Apps with Ads:** Across all 34 categories, paid apps and apps with clean monetization had equal or better user engagement than free apps packed with annoying ads.

7 Conclusion and References

7.1 Conclusion

This case study explored whether the factors driving app store success are universal or category-specific using a clean dataset of 12,669 Google Play apps. After handling real-world issues like Google’s switch

to Android App Bundles, filtering for reliable apps ($N \geq 50$), and creating a fair category-relative target, we tested both store-wide models and 34 individual category models.

Our findings show significant proof that app success is heavily category-dependent. Things like pricing tiers and update frequency behave in completely different ways depending on the type of app. Developers who tailor their strategy to their specific category will make much smarter product decisions than those following generic advice.

7.2 Limitations and Future Work

- **Snapshot Data:** The dataset is a point-in-time snapshot. Future projects could track apps over several months to see how individual updates change user ratings over time.
- **No Review Text:** We only used numerical metadata. Adding text sentiment analysis on user reviews would give deeper clues into specific complaints.

References

- [1] Ama, N. A. (2025). Machine Learning Based-prediction of Health Application Effectiveness on Google Play Store. *Trends in Computer Science and Information Technology*, 10(3), 081–088. <https://doi.org/10.17352/tcsit.000101>
- [2] IEEE Xplore. (2024). App Rating Prediction Using Machine Learning. *IEEE International Conference on Computing and Data Science*, Document ID: 10425706, 1–6. <https://doi.org/10.1109/ICCD60340.2024.10425706>
- [3] Aljrees, T., Umer, M., Saidani, O., Almuqren, L., Ishaq, A., Alsubai, S., Eshmawi, A. A., & Ashraf, I. (2024). Contradiction in text review and apps rating: prediction using textual features and transfer learning. *PeerJ Computer Science*, 10, e1722. PMCID: PMC10773744. PMID: 38196956. <https://doi.org/10.7717/peerj-cs.1722>
- [4] Android Developers. (2023). *About Android App Bundles and Dynamic Delivery*. Google Developers Documentation. <https://developer.android.com/guide/app-bundle>
- [5] Google Play Store. (2026). *Public Application Marketplace Listings Directory*. Alphabet Inc. Accessed September 2026. <https://play.google.com/store/apps>